Whack-a-Mole in Qt

A plain-language guide to the whole project: what every file does, how the code works, and what to say when you present it.

How to use this guide. Read it once from top to bottom. Each section explains one key moment of the program in simple words, then shows the real code from the project. You do not need to know anything about GUI programming beforehand. Terms in `code font` are names from the code. Section 14 is a glossary if a word is new.

#	Section	What you will understand
1	The big picture	What the program does and the one idea that explains all GUI programs
2	The files	Which file does what
3	main.cpp	How the program starts
4	game_window.h	The class declaration: signals, slots, variables
5	Building the window	Widgets, group boxes, layouts
6	Connections	How clicks and timers reach your functions
7	The game logic	Timers, mole movement, score, game over
8	Dialogs	Name, rules, stop question, game over
9	Enabling and disabling	Why buttons switch on and off
10	ClockLabel and the .pro file	The clock and the build file
11	Lessons map	Which lesson each feature comes from
12	Questions your teacher may ask	Short answers
13	Presenting and changing the game	Demo script and easy edits
14	Glossary	Every term in one line

1. The big picture

What the program does

Whack-a-Mole is a small game. A 3x3 grid of buttons is on the screen. A "mole" appears on one random button. The player clicks it to get a point, and the mole jumps to another button. When the time runs out, a window shows the score and asks whether to play again.

Before the game the player can type a name, choose the difficulty (how fast the mole moves), choose the game length (10 to 120 seconds), and switch two options on or off.

The one idea behind every GUI program

Key idea: event, function, change, wait
A normal console program runs from the first line to the last line. A GUI program does not. It starts, shows the window, and then **waits**. This waiting is called the **event loop**.
When something happens (a click, a key press, a timer ticking) Qt calls one of your functions. That function changes some numbers and some widgets. Then the program goes back to waiting.
So to understand this project you only need to answer: **which event calls which function, and what does that function change?** Everything in this guide is built around that question.

Five Qt words you need

Word	Meaning in simple terms	Example in this project
Widget	Anything you can see in a window: button, label, text box, the window itself.	`QPushButton`, `QLabel`, `QSpinBox`, `GameWindow`
Layout	An invisible helper that places widgets next to or under each other and resizes them when the window changes size.	`QHBoxLayout`, `QVBoxLayout`, `QGridLayout`
Signal	A message a widget sends when something happened to it. You do not write it, you only use it.	`clicked()`, `valueChanged(int)`, `timeout()`
Slot	A normal function of your class that runs when a signal it is connected to is sent.	`startGame()`, `onHoleClicked(int)`
connect	The line that joins a signal to a slot: "when this happens, call that".	`connect(m_startButton, &QPushButton::clicked, this, &GameWindow::startGame);`

2. The files

File	What is inside	Size
WhackAMole.pro	The project file. Tells qmake which files to compile and which Qt parts to use.	tiny
main.cpp	Creates the application and the window, shows it, starts the event loop.	small
game_window.h	Declares the class `GameWindow`: its functions, signals, slots and variables.	medium
game_window.cpp	The real work: builds the window and contains all the game logic.	biggest
clock_label.h / .cpp	A label that shows the current time and updates every second. This is your homework from Lesson 7.	small

How they connect. `main.cpp` creates one `GameWindow`. The `GameWindow` creates one `ClockLabel` and puts it in the top-right corner. Nothing else is needed.

```
main.cpp  --creates-->  GameWindow  --contains-->  buttons, labels, spin box, ...
                                    --contains-->  ClockLabel (top-right clock)
                                    --owns------>  2 x QTimer (mole timer, countdown timer)
```

3. main.cpp - how the program starts

```
#include <QApplication>
#include <QTimer>
#include "game_window.h"

int main(int argc, char* argv[])
{
    QApplication app(argc, argv);

    GameWindow* window = new GameWindow;
    window->show();

    QTimer::singleShot(0, window, &GameWindow::askPlayerName);

    return app.exec();
}
```

Line	What it does
`QApplication app(argc, argv);`	Creates the Qt application object. Every Qt GUI program needs exactly one, and it must exist before any widget.
`new GameWindow`	Creates our window. Its constructor builds all the widgets (section 5).
`window->show();`	Windows are hidden until you call `show()`.
`QTimer::singleShot(0, ...)`	Says: "call `askPlayerName()` once, as soon as the event loop starts". This way the main window is already visible behind the name dialog.
`return app.exec();`	Starts the event loop. The program now waits for events. `exec()` only returns when the last window is closed, and then the program ends.

`main` is almost the same as the "Hello Qt" and "Quit App" examples in your Lesson 1 and 2 slides. The only difference is that we show our own class instead of a single `QLabel`.

4. game_window.h - the class declaration

A header file lists what a class contains, but not how it works. Here are the important parts of `game_window.h` (shortened):

```
class QLabel;  class QPushButton;  class QTimer;  ...    // forward declarations

class GameWindow : public QWidget
{
    Q_OBJECT

public:
    GameWindow(QWidget* parent = nullptr);

public slots:
    void askPlayerName();

signals:
    void scoreChanged(int score);
    void gameOver(int finalScore);

private slots:
    void startGame();                 void stopGame();
    void onHoleClicked(int index);    void onMoleTimeout();
    void onCountdownTick();           ...

private:
```

```
    void createWidgets();  void createLayout();  void makeConnections();
    void moveMole();       void hideMole();      void resetBoard();     ...

    enum { ROWS = 3, COLS = 3, HOLES = ROWS * COLS };
    QPushButton* m_holes[HOLES];
    QTimer* m_moleTimer;   QTimer* m_countdownTimer;
    int  m_score, m_bestScore, m_timeLeft, m_moleIndex;
    bool m_running;
};
```

Part	Why it is there
`class QLabel;` (forward declarations)	Tells the compiler "this class exists" without including the whole header. It keeps compile times short. The Find Dialog files from your course do the same.
`: public QWidget`	`GameWindow` **is** a widget (a window). It inherits everything a widget can do.
`Q_OBJECT`	A macro that switches on signals and slots for this class. Without it, `connect` would not work. A Qt tool called **moc** (meta-object compiler) reads this macro and generates extra code (Lesson 3).
`signals:`	Our own messages. We only declare them. Qt writes their body. We send one with `emit`.
`slots:`	Normal functions that can be connected to signals. `private slots` can be used by `connect` inside the class but not called from outside.
`createWidgets` / `createLayout` / `makeConnections`	The window setup is split into three steps so the code stays readable. Same pattern as your Find Dialog.
`enum { ROWS, COLS, HOLES }`	Named constants. If you change `ROWS` and `COLS`, the whole grid changes (section 13).
`m_` prefix	Just a naming habit: `m_` means "member variable", a variable that belongs to the class.
`m_moleIndex`	Which hole (0 to 8) currently has the mole, or -1 when there is none.
`m_running`	`true` while a game is being played. Many functions check it first.

5. Building the window

The constructor of `GameWindow` builds the window in three steps, always in this order:

```
GameWindow::GameWindow(QWidget* parent)
    : QWidget(parent), m_score(0), m_bestScore(0),
      m_timeLeft(30), m_moleIndex(-1), m_running(false)
{
    createWidgets();      // 1. create the top/bottom widgets and the timers
    createLayout();       // 2. create the group boxes and arrange everything
    makeConnections();    // 3. connect signals to slots

    setWindowTitle("Whack-a-Mole");
    resize(760, 440);
    ...
    resetBoard();         // start in the "idle" state
}
```

Why the order matters. `makeConnections()` needs every widget to exist already. Some widgets (name box, radio buttons, spin box, the 9 holes) are created inside the `create...Group()` functions that `createLayout()` calls. So connections must come last.

What the window looks like

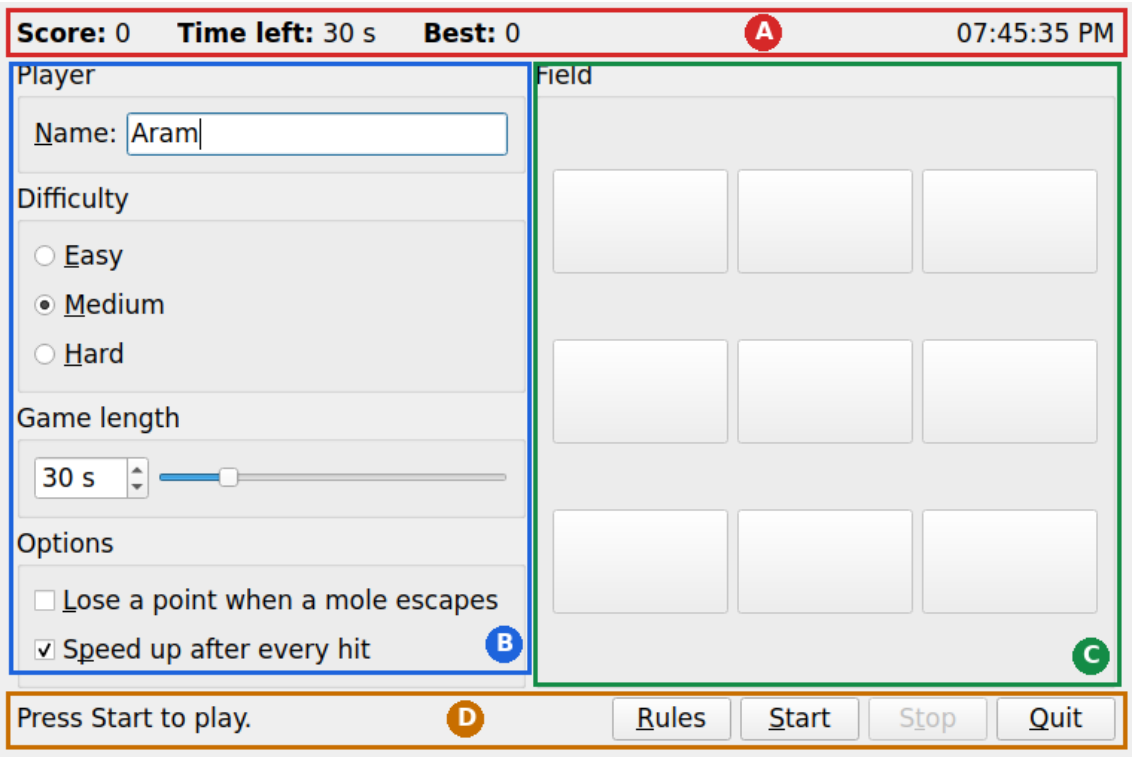


Figure 1. The window before a game. Red A: top bar. Blue B: settings. Green C: the field. Orange D: bottom bar.

How the layouts are nested

A layout can contain widgets **and other layouts**. The window is built like boxes inside boxes. Read the picture from the outside in:

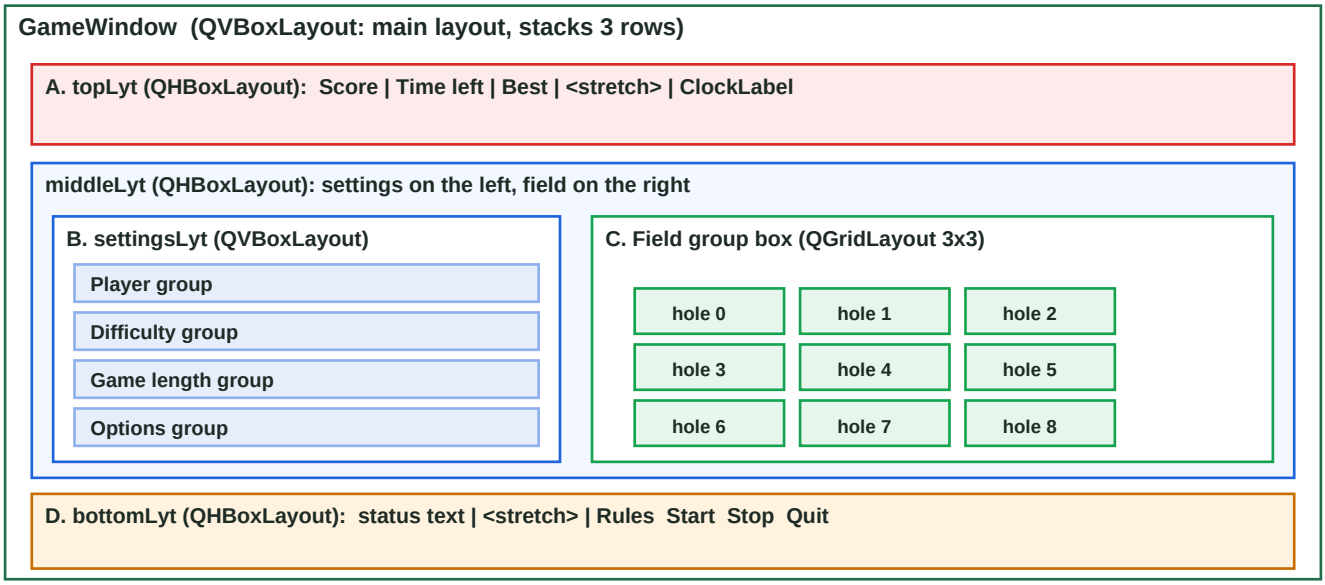


Figure 2. Nested layouts. The outer vertical layout stacks A, the middle part and D.

Layout	Arranges	Used for
`QHBoxLayout`	left to right	top bar, bottom bar, settings next to field, spin box next to slider
`QVBoxLayout`	top to bottom	the whole window, the settings column, the radio buttons
`QGridLayout`	rows and columns	the 3x3 field of mole holes
`addStretch()`	empty springy space	pushes the clock to the right and the buttons to the right

Group boxes and radio buttons

A `QGroupBox` is a framed container with a title. Each settings block is one group box. Radio buttons placed in the same group box automatically work as a set: choosing one switches the others off (Lesson 5). Check boxes are independent, so any of them can be on.

```
QGroupBox* GameWindow::createDifficultyGroup()
{
    QGroupBox* group = new QGroupBox("Difficulty");

    m_easyRadio   = new QRadioButton("&Easy");
    m_mediumRadio = new QRadioButton("&Medium");
    m_hardRadio   = new QRadioButton("&Hard");
    m_mediumRadio->setChecked(true);           // Medium is selected at the start

    QVBoxLayout* lyt = new QVBoxLayout;        // put the 3 buttons under each other
    lyt->addWidget(m_easyRadio);
    lyt->addWidget(m_mediumRadio);
    lyt->addWidget(m_hardRadio);
    group->setLayout(lyt);                     // give the layout to the group box
    return group;
}
```

The `&` before a letter creates a keyboard shortcut: `&Easy` can be reached with Alt+E. The letter is underlined on screen. The name box uses `setBuddy`, so pressing Alt+N jumps into the text field (the same trick as in the Find Dialog).

The field of holes

The 9 buttons are made in a loop and placed into a `QGridLayout`. The grid wants a row number and a column number, but the loop only has a counter `i` from 0 to 8. Division and remainder turn one into the other:

```
QGridLayout* grid = new QGridLayout;
for (int i = 0; i < HOLES; ++i) {
    m_holes[i] = new QPushButton;
    m_holes[i]->setMinimumSize(100, 70);
    m_holes[i]->setEnabled(false);                 // empty hole = disabled button
    grid->addWidget(m_holes[i], i / COLS, i % COLS);   // row = i / 3, column = i % 3
}
```

	column 0	column 1	column 2
row 0	hole 0	hole 1	hole 2
row 1	hole 3	hole 4	hole 5
row 2	hole 6	hole 7	hole 8

Example: hole 5 has row `5 / 3 = 1` and column `5 % 3 = 2`. This is the same number the game uses in `m_moleIndex`.

6. Connections - how events reach your code

`makeConnections()` is the list of all "when X happens, call Y" rules. The project uses all three ways to write `connect` that you learned in Lesson 2.

```
// 1. Old style: text macros SIGNAL() and SLOT()
connect(m_quitButton, SIGNAL(clicked()), this, SLOT(close()));

// 2. Modern style: function pointers (checked by the compiler)
connect(m_startButton, &QPushButton::clicked, this, &GameWindow::startGame);

// 3. Lambda style: a small function written right inside connect
for (int i = 0; i < HOLES; ++i) {
    connect(m_holes[i], &QPushButton::clicked, this, [this, i]() {
        onHoleClicked(i);
    });
}
```

Why the holes need a lambda

`clicked()` carries no information about **which** button was clicked. But the game must know whether the player clicked the hole with the mole. The lambda solves this: each of the 9 buttons gets its own little function that remembers its own number `i` and calls `onHoleClicked(i)`. In `[this, i]` the square brackets are the **capture list**: `this` lets the lambda call our class function, and `i` copies the current loop number into the lambda.

The full list of connections

Signal (what happened)	Slot (what runs)	Result
Quit button `clicked()`	`close()`	Window closes, program ends
Start button `clicked()`	`startGame()`	A game begins
Stop button `clicked()`	`stopGame()`	Asks if you want to stop
Rules button `clicked()`	`showRules()`	Shows the rules message
Name box `textChanged(QString)`	`checkName()`	Start is enabled only if the name is not empty
Spin box `valueChanged(int)`	slider `setValue(int)`	Slider follows the spin box
Slider `valueChanged(int)`	spin box `setValue(int)`	Spin box follows the slider
Spin box `valueChanged(int)`	`onLengthChanged(int)`	"Time left" label shows the chosen length
Mole timer `timeout()`	`onMoleTimeout()`	The mole was too slow and moves
Countdown timer `timeout()`	`onCountdownTick()`	One second less; end the game at 0
Each hole `clicked()` (lambda)	`onHoleClicked(i)`	Score check
Our `scoreChanged(int)`	`updateScoreLabel(int)`	Score label text updates
Our `gameOver(int)`	`showGameOverDialog(int)`	Result window opens

Spin box and slider linked in both directions. Each one sends `valueChanged` to the other's `setValue`. This looks like it could loop forever, but Qt only sends `valueChanged` when the value really changes, so the loop stops after one step (this is the "Enter your age" example from Lesson 3).
One signal, many slots. The spin box's `valueChanged` goes to two places (the slider and `onLengthChanged`). Lesson 2 says exactly this: a signal can be connected to many slots.

7. The game logic

The numbers the game remembers

Variable	Meaning	Changes when
`m_running`	Is a game in progress?	`true` in `startGame`, `false` in `resetBoard`
`m_score`	Points in this game	+1 on a hit; -1 on an escape if that option is on
`m_bestScore`	Best score since the program started	Game over, if the score is higher
`m_timeLeft`	Seconds remaining	-1 every second
`m_moleIndex`	Hole with the mole (0 to 8), or -1	`moveMole()` and `hideMole()`

Two timers do all the timing

Timer	Interval	Job
`m_moleTimer`	Easy 1200 ms, Medium 850 ms, Hard 550 ms	If it fires, the player was too slow: the mole moves
`m_countdownTimer`	always 1000 ms	Counts the game time down by one second

Using a timer always follows the three steps from Lesson 7: create the `QTimer`, connect `timeout()` to a slot, call `start()`. Both timers are created with `this` as their parent so Qt deletes them together with the window.

The life of one game

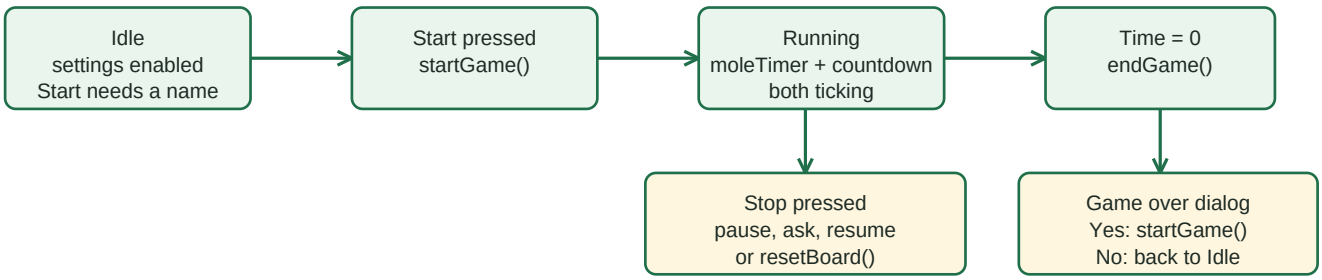


Figure 3. The states of the game. Yellow boxes are things that can happen during a game or when the time ends.

Step 1: Start is pressed - startGame()

```
void GameWindow::startGame()
{
    m_running = true;
    m_score = 0;
    emit scoreChanged(m_score);          // our own signal -> score label shows 0

    m_timeLeft = m_lengthSpin->value();  // length chosen in the spin box
    updateTimeLabel();

    setSettingsEnabled(false);           // lock all settings during the game
    m_statusLabel->setText("Game is running - hit the moles!");

    m_moleTimer->start(difficultyInterval());   // Easy / Medium / Hard speed
    m_countdownTimer->start();                  // 1 second ticks
    moveMole();                                 // first mole appears at once
}
```

`difficultyInterval()` asks the radio buttons which one is selected with `isChecked()` and returns 1200, 850 or 550 milliseconds.

Step 2: a mole appears - moveMole()

```
void GameWindow::moveMole()
{
    int oldIndex = m_moleIndex;
    hideMole();                          // empty the old hole (text "", disabled)

    int newIndex = oldIndex;
    while (newIndex == oldIndex)         // pick a hole different from the last one
        newIndex = QRandomGenerator::global()->bounded(static_cast<int>(HOLES));

    m_moleIndex = newIndex;
    m_holes[m_moleIndex]->setText("(o_o)\nMOLE!");
    m_holes[m_moleIndex]->setEnabled(true);    // only the mole hole can be clicked
}
```

A hole with a mole is simply a button that has text and is enabled. An empty hole is a button with no text that is disabled (greyed out). There is no drawing code at all.

Step 3a: the player clicks - onHoleClicked(index)

```
void GameWindow::onHoleClicked(int index)
{
    if (!m_running || index != m_moleIndex)
        return;                                  // ignore anything that is not a real hit

    ++m_score;
    emit scoreChanged(m_score);

    if (m_speedUpCheck->isChecked()) {
        int faster = m_moleTimer->interval() * 93 / 100;   // 7 percent faster
        if (faster < 300) faster = 300;                    // but never faster than 300 ms
        m_moleTimer->start(faster);                        // restarts the timer
    } else {
        m_moleTimer->start();                              // restart with the same interval
    }
    moveMole();
}
```

Calling `start()` on a running timer **restarts** it. That is important: after a hit the new mole gets a full fresh time, instead of inheriting what was left of the old one. With Medium (850 ms) and speed-up on, the interval goes 850, 790, 734, 682 and so on, down to 300 ms.

Step 3b: the player is too slow - onMoleTimeout()

```
void GameWindow::onMoleTimeout()
{
    if (!m_running) return;
    if (m_penaltyCheck->isChecked() && m_score > 0) {   // optional penalty, never below 0
        --m_score;
        emit scoreChanged(m_score);
    }
    moveMole();
}
```

Step 4: every second - onCountdownTick()

```
void GameWindow::onCountdownTick()
{
    --m_timeLeft;
    updateTimeLabel();
    if (m_timeLeft <= 0)
        endGame();
}

void GameWindow::endGame()
{
    resetBoard();                // stop both timers, empty the field, unlock settings
    m_statusLabel->setText("Game over. Press Start to play again.");
    emit gameOver(m_score);      // -> showGameOverDialog(finalScore)
}
```

Why use our own signals? `scoreChanged` and `gameOver` could be replaced by direct function calls, and the game would work the same. We use signals because that is the Qt way and because the sender does not need to know who listens (the Find Dialog from your course does this with `findNext` and `findPrev`).

8. Dialogs

A dialog is a small window that asks something or tells something. The project uses four dialogs from Lesson 6. All of them are **modal**: the rest of the program is blocked until the dialog is closed.

When	Function	Qt class / function	What the user does
Right after start	`askPlayerName()`	`QInputDialog::getText`	Types a name, presses OK
Rules pressed	`showRules()`	`QMessageBox::information`	Reads, presses OK
Stop pressed	`stopGame()`	`QMessageBox::question`	Yes or No
Time is over	`showGameOverDialog()`	`QMessageBox` object + `exec()`	Yes (play again) or No

Game over dialog - the property-based way

```
QMessageBox msgBox(this);
msgBox.setWindowTitle("Game over");
msgBox.setIcon(QMessageBox::Information);
msgBox.setText(QString("Time's up, %1!").arg(m_nameEdit->text().trimmed()));
msgBox.setInformativeText(result);                // score, record or best score
msgBox.setStandardButtons(QMessageBox::Yes | QMessageBox::No);
msgBox.setDefaultButton(QMessageBox::Yes);

int ret = msgBox.exec();                          // waits here until a button is pressed
switch (ret) {
case QMessageBox::Yes:  startGame();  break;      // play again
case QMessageBox::No:
default:                m_timeLeft = m_lengthSpin->value();
                        updateTimeLabel();  break;
}
```

`exec()` returns which button was pressed, so we can react with a `switch`, exactly like the "Do you want to save your changes?" example in Lesson 6. Before the dialog opens, the function compares the score with `m_bestScore` and updates the "Best" label if it is a new record.

The Stop question - an important detail

```
void GameWindow::stopGame()
{
    if (!m_running) return;

    m_moleTimer->stop();              // pause both timers FIRST
    m_countdownTimer->stop();

    int answer = QMessageBox::question(this, "Stop game",
                    "Do you really want to stop the current game?",
                    QMessageBox::Yes | QMessageBox::No, QMessageBox::No);
    if (answer == QMessageBox::Yes) {
        resetBoard();  ...            // really stop
    } else {
        m_moleTimer->start();         // continue: resume both timers
        m_countdownTimer->start();
    }
}
```

Why the timers are stopped before the question. A modal dialog blocks the buttons, but timers keep running in the background. Without stopping them, the game clock would keep counting down and the mole would keep moving while the player is reading the question.

9. Enabling and disabling widgets

Many parts of the interface are switched on and off to stop the player from doing something that makes no sense. This is the same idea as the Find button in the Find Dialog, which stays disabled until there is text.

Widget	Enabled when	Where it is controlled
Start button	A name is typed and no game is running	`checkName()`, `setSettingsEnabled()`
Stop button	A game is running	`setSettingsEnabled()`
All four settings group boxes	No game is running	`setSettingsEnabled()`
A hole button	It is the one with the mole	`moveMole()` and `hideMole()`

```
void GameWindow::setSettingsEnabled(bool enabled)
{
    m_playerGroup->setEnabled(enabled);        // disabling a group box also
    m_difficultyGroup->setEnabled(enabled);    // disables everything inside it
    m_lengthGroup->setEnabled(enabled);
    m_optionsGroup->setEnabled(enabled);

    m_startButton->setEnabled(enabled && !m_nameEdit->text().trimmed().isEmpty());
    m_stopButton->setEnabled(!enabled);        // Stop is the opposite of the settings
}
```

`resetBoard()` calls `setSettingsEnabled(true)` and `startGame()` calls `setSettingsEnabled(false)`. That one function is the switch between the "idle" and the "playing" look of the window.

10. ClockLabel and the .pro file

ClockLabel - your Lesson 7 homework

```
ClockLabel::ClockLabel(QWidget* p) : QLabel(p)
{
    QTimer* tmr = new QTimer(this);
    tmr->setInterval(1000);                          // every second
    connect(tmr, SIGNAL(timeout()), this, SLOT(updateTime()));
    tmr->start();
    updateTime();                                    // show the time immediately
}

void ClockLabel::updateTime()
{
    QTime t = QTime::currentTime();
    setText(t.toString("hh:mm:ss AP"));              // for example 07:45:35 PM
}
```

`ClockLabel` **is a** `QLabel` that carries its own timer. In the game we just create one and put it in the top bar. We changed the original interval from 3000 to 1000 ms so it really shows seconds, and gave the timer a parent so it is cleaned up correctly. The format letters (`hh`, `mm`, `ss`, `AP`) are from your Lesson 7 table.

WhackAMole.pro

```
TEMPLATE = app                                  # build an application (not a library)
TARGET = WhackAMole                             # name of the program
QT = core gui
greaterThan(QT_MAJOR_VERSION, 4): QT += widgets # widgets are needed for Qt 5 and Qt 6
CONFIG += c++11

SOURCES += main.cpp game_window.cpp clock_label.cpp
HEADERS += game_window.h clock_label.h
```

`qmake` reads this file and produces the build instructions. The headers are listed too because they contain `Q_OBJECT`, so the moc tool must process them.

11. Which lesson each feature comes from

Feature in the game	Lesson
`.pro` file, `QApplication`, `app.exec()`, showing a widget	Lesson 1
Quit button with `SIGNAL/SLOT`, Start/Stop with `&QPushButton::clicked`, holes with a lambda	Lesson 2
Bold text in labels such as `<b>Score:</b>`	Lesson 2 (Hello Qt example)
Game length: `QSpinBox` and `QSlider` linked both ways	Lesson 3
`Q_OBJECT`, own slots and signals, moc	Lesson 3 / Find Dialog
`QHBoxLayout`, `addWidget`, `setLayout`	Lesson 3
`QVBoxLayout`, `addStretch()`	Lesson 5 / Find Dialog
Difficulty radio buttons, options check boxes, group boxes, `isChecked()`	Lesson 5
The 3x3 field with `QGridLayout`	Lesson 5
Name dialog `QInputDialog::getText`	Lesson 6
Rules, Stop question, Game over (`information`, `question`, `QMessageBox` + `exec()` + `switch`)	Lesson 6
Mole timer and countdown timer: `QTimer`, `timeout()`, `start()`, `stop()`, `setInterval`	Lesson 7
Clock label: `QTime::currentTime()`, `toString(format)`	Lesson 7
`createWidgets / createLayout / makeConnections`, `setBuddy`, `&` shortcuts, `addStretch`, enable/disable, own signals with `emit`	Find Dialog files

Small things that go slightly beyond the lessons

Be ready to explain these in one sentence if the teacher asks. They are tiny helpers, not new concepts.

Used in the code	What it is
`QTimer::singleShot(0, ...)`	A timer that fires once. Lesson 7 mentions the `singleShot` property.
`QRandomGenerator::global()->bounded(9)`	Gives a random whole number from 0 to 8.
`QString("Score: %1").arg(n)`	Puts the number `n` where `%1` is in the text.
`trimmed()`	Removes spaces at the beginning and the end of a text.
`setSuffix(" s")`, `setMaxLength(20)`, `setMinimumSize(w, h)`, `addSpacing(n)`	Small settings: text after the number, longest allowed name, smallest button size, empty gap.
`: QWidget(parent), m_score(0), ...` after the constructor	C++ initializer list: sets starting values.

12. Questions your teacher may ask

Q: What is the difference between a signal and a slot?

A: A signal is a message that a widget sends when something happens, for example `clicked()`. A slot is an ordinary function that is called when a connected signal is sent. `connect` links them.

Q: What does `Q_OBJECT` do?

A: It enables signals and slots (and other meta-object features) for the class. The moc tool reads it and generates the extra code. Without it `connect` does not work for our class.

Q: What does `app.exec()` do?

A: It starts the event loop. The program waits for events and calls our functions. It returns when the last window is closed.

Q: How does the program know which hole was clicked?

A: Each hole is connected through a lambda that remembers its own number `i` and calls `onHoleClicked(i)`. The function compares `i` with `m_moleIndex`.

Q: Why do you have two timers?

A: They do two different jobs at different speeds: the mole timer (550 to 1200 ms, depends on difficulty) moves the mole, the countdown timer (1000 ms) counts the game time.

Q: Why do you stop the timers when the Stop question appears?

A: A modal dialog does not stop timers. We pause them so the game does not continue behind the question.

Q: Why `new` without `delete`?

A: In Qt, widgets that are put into a layout or created with a parent become children of that parent, and a parent deletes its children automatically. The main window itself lives until the program ends, and the operating system frees its memory then.

Q: Why are empty holes disabled buttons?

A: A disabled button cannot be clicked, so only the hole with the mole reacts. It also looks greyed out, which looks like an empty hole, without any drawing code.

Q: Why split the setup into three functions?

A: It keeps the constructor short and the code readable: create the widgets, arrange them, then connect them. It is the same structure as the Find Dialog.

Q: Why does the spin box and slider connection not loop forever?

A: Qt sends `valueChanged` only when the value actually changes. After one round both have the same value and nothing more is sent.

13. Presenting and changing the game

A 5-minute presentation plan

- **Run the game (1 min).** Show the name dialog, the settings, then play a few seconds so the teacher sees the mole, the score and the clock.
- **Show the structure (1 min).** Open the project tree: `main.cpp`, `game_window`, `clock_label`.
- **Show the window code (1 min).** In `game_window.cpp` show `createLayout()` and say "boxes inside boxes" (Figure 2). Mention group boxes, radio buttons and the grid.
- **Show the connections (1 min).** Open `makeConnections()` and point out the three styles of `connect`, including the lambda.
- **Show the game logic (1 min).** Show `startGame`, `moveMole`, `onHoleClicked`, and the two timers. Say: "one timer moves the mole, one counts the seconds."

Easy changes you can make and show

Change	Where	How
Make Hard even harder	`difficultyInterval()` in `game_window.cpp`	Change `550` to `400`
Bigger field (4x4)	`game_window.h`	Change `ROWS = 3, COLS = 3` to `ROWS = 4, COLS = 4`. Everything else adapts automatically. Tested: it builds.
Different mole text	`moveMole()`	Change the string `"(o_o)\nMOLE!"`
Longer maximum game	`createLengthGroup()`	Change `120` in both `setRange(10, 120)` lines
Different start length	`createLengthGroup()`	Change `30` in `setValue(30)` (both lines)
Window title	constructor	Change `setWindowTitle("Whack-a-Mole")`

After any change, press the green Run button again. Qt Creator rebuilds automatically.

If the build fails in Qt Creator

- Check that you opened `WhackAMole.pro` from inside Qt Creator (File, Open File or Project).
- Check that you selected a **Desktop Qt ... MinGW 64-bit** kit when it asked to Configure Project.
- Try Build, then Run qmake, then Run again.
- Keep the project in a short folder without spaces or Armenian letters, for example `C:\Projects\WhackAMole`.

14. Glossary

Term	Meaning
Widget	Any visible element of a window (button, label, box). A window is also a widget.
Layout	Object that places widgets and resizes them automatically.
Signal / slot	Message sent by a widget / function that reacts to it.
`connect`	Joins a signal to a slot.
`emit`	Sends one of our own signals.
Event loop	The waiting loop started by `app.exec()`. It receives events and calls our functions.
moc	Meta-object compiler. Reads `Q_OBJECT` classes and generates the code that makes signals and slots work.
Parent / child	A widget created with (or placed into) a parent is deleted automatically together with it.
Modal dialog	A window that blocks the rest of the program until it is closed.
Lambda	A small unnamed function written directly in the code, such as `[this, i]() { ... }`.
Capture list `[this, i]`	The variables a lambda takes with it from the surrounding code.
Single-shot timer	A timer that fires only once instead of repeating.
Interval	Time between two `timeout()` signals, in milliseconds (1000 ms = 1 second).
Kit (Qt Creator)	The combination of Qt version and compiler used to build, for example Desktop Qt 5.15 MinGW 64-bit.
qmake / .pro	The tool and the project file that describe how to build the program.